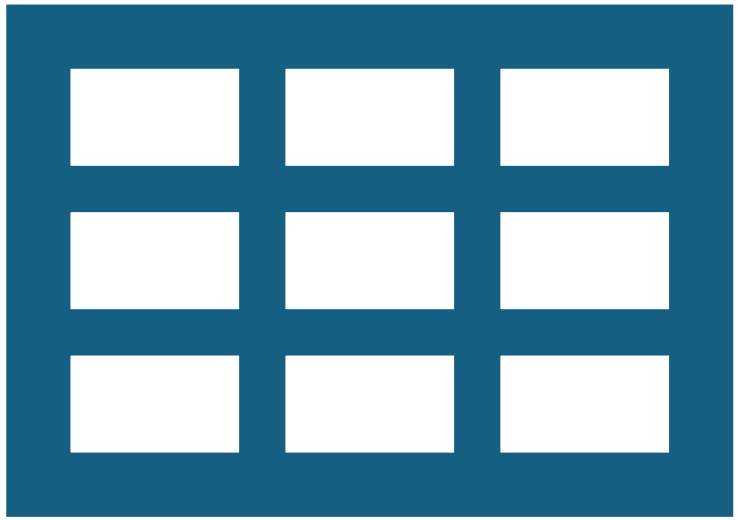


Course 10

NoSQL – Wide Column

Wide Column Store

Definition: A NoSQL database that organizes data into columns with dynamic rows.

The Hybrid Nature: Sits between Key-Value stores and Relational Databases.

Key Concept: A "two-dimensional key-value store."

Structure:

- Row Key (Primary Index)
- Column Family (Group of related columns)
- Column Qualifier (Dynamic key within the row)
- Timestamp (Versioning)

Row Oriented vs Wide Column

Row-Oriented (RDBMS):	Data stored sequentially by row (e.g., CSV style). Good for OLTP (fetching one user's full profile). Example: •[Block 1: id, name, email, city, theme_style] values: [1,'Alice','alice@email.com','New York','light'] •[Block 2: id, name, email, city, theme_style] values: [2,'Mary','mary@xyz.abc','Frankfurt',NULL]
Wide Column:	Data stored by column families; sparse. Rows can have different columns. Example: •Row 1 -> {name: "Alice", email: "alice@email.com", city: "New York", theme_style: "light"} •Row 2 -> {name: "Mary", email: "mary@xyz.abc", city: "Frankfurt"}

id	name	email	city	theme_style
1	Alice	alice@email.com	New York	light
2	Bob	bob@comp.net	London	dark
3	Mary	mary@xyz.abc	Frankfurt	



Row 1

name	email	city	theme_style
Alice	alice@email.com	New York	light

Row 2

name	email	city	theme_style
Bob	bob@comp.net	London	dark

Row 3

name	email	city
Mary	mary@xyz.abc	Frankfurt

Recap: Big Table

Google published the "Bigtable" paper in 2006.

Storing petabytes of data across thousands of commodity servers (Web Indexing, Google Earth).

A sparse, distributed, persistent multi-dimensional sorted map.

Inspired Apache HBase (Hadoop) and Apache Cassandra (Facebook).

Characteristics



Schema-Flexibility: Columns can be added on the fly without locking the table.



High Availability: Designed to tolerate node failures without downtime.



Scalability: Horizontal scaling (scale-out) using commodity hardware.



Distributed: Data is partitioned across multiple nodes (Sharding).

Distributed Architectures

- Master-Slave (HBase, Bigtable):
  - Master: Handles metadata and coordination.
  - Slaves (RegionServers): Handle data IO.
  - Pro: Strong consistency.
  - Con: Master is a single point of failure (conceptually).

- Peer-to-Peer / Masterless (Cassandra, ScyllaDB):
  - All nodes are equal.
  - Any node can handle a request (Coordinator Node).
  - Pro: Max availability.
  - Con: Eventual consistency complexity.

CAP/PACELC

CAP: Consistency, Availability, Partition Tolerance. Pick two.

Wide Column Positioning:

- AP (Availability + Partition Tolerance): Cassandra, ScyllaDB.
- CP (Consistency + Partition Tolerance): HBase, Bigtable.

Tunable Consistency: The ability to trade latency for consistency per-query (e.g., QUORUM vs. ONE).

Data Distribution

Consistent Hashing

How to distribute data evenly without rehashing everything when a node is added?

The *Token Ring*:

- Data is hashed (e.g., Murmur3) to a token.
- Nodes own ranges of tokens on a circular ring.
- Virtual Nodes (VNodes): Physical nodes own multiple small ranges to improve load balancing.

Inter Node Communication



Gossip Protocol



A peer-to-peer communication protocol where nodes periodically exchange state information.



Failure detection (Phi Accrual Failure Detector).



Schema propagation.



Topology changes.



Conceptualy: "I tell two friends, they tell two friends..." - The cluster state converges quickly.

Read/Write Paths

Write

Log-Structured Merge Tree: Optimized for high write throughput.

Step 1: Commit Log: Append-only file on disk (Durability).

Step 2: MemTable: In-memory sorted data structure.

Step 3: Flush: When MemTable is full, it is flushed to disk as an SSTable (Sorted String Table).

Step 4: SSTables are immutable.

Read

Challenge: Data might exist in the MemTable or across multiple SSTables.

Step 1: Check MemTable.

Step 2: Check Row Cache (if enabled).

Step 3: Check Bloom Filters (Probabilistic structure: "Is the data definitely not here?").

Step 4: Scan Key Cache / Partition Summary.

Step 5: Fetch from SSTables and merge results (Last Write Wins).

Compaction

Write path creates many small SSTables and "Tombstones" (deletion markers).

Compaction: Background process that merges SSTables.

- Purges deleted data (Tombstones).
- Combines fragments into larger files for faster reads.
- Strategies: Size-Tiered (Write heavy), Leveled (Read heavy).

Conceptual Data Model

Keyspace

Container for tables (roughly maps to Database in SQL).

- Column Family / Table: Container for rows.
- Row: Identified by a unique Primary Key.
- Column: Name, Value, Timestamp.

Primary Key

Partition vs. Clustering

The most critical concept in wide column modeling.

Partition Key:

- Determines which node stores the data.
- Responsible for data distribution.

Clustering Key:

- Determines how data is sorted on the disk within the partition.
- Responsible for range queries.

Design

Query-Driven

RDBMS Approach: Normalize data to remove redundancy -> Join on read.

Wide Column Approach: Denormalize data based on access patterns -> Duplicate on write.

Rule: Design your tables to satisfy your queries. Do not model entities first.

Motto: "Space is cheap, reads are expensive."

Use Cases



- High Write Throughput: IoT sensor data, Logging, Clickstreams.
- Time Series Data: Financial ticks, metrics, history.
- Geographically Distributed Apps: Multi-region replication.
- Personalization Engines: User profiles and activity streams.



- ACID Transactions: If you need banking-level atomic transfers between tables.
- Ad-Hoc Queries: "Find users where Age > 25 AND City = 'NY'" (Requires full table scans = slow).
- Joins: Wide column stores generally do not support server-side joins.
- Small datasets: The overhead of a distributed system isn't worth it for < 50GB.

Products

Apache Cassandra: masterless, linear scalability, multi-datacenter replication

Apache Hbase: open source of BigTable, runs on HDFS (Hadoop), strong consistency

ScyllaDB: rewrite of Cassandra in C++, shared-nothing, 10x performance of Cassandra

Google Bigtable: decouples compute from storage, scale to petabytes, Hbase API compatible

Rank			DBMS	Database Model	Score		
Nov 2025	Oct 2025	Nov 2024			Nov 2025	Oct 2025	Nov 2024
1.	1.	1.	Apache Cassandra	Wide column, Multi-model	102.71	-2.45	+5.00
2.	2.	↑ 3.	Microsoft Azure Cosmos DB	Multi-model	22.55	-0.16	-1.40
3.	3.	↓ 2.	Apache HBase	Wide column	19.32	-0.85	-5.93
4.	4.	↑ 5.	ScyllaDB	Wide column, Multi-model	4.13	+0.28	+0.25
5.	5.	↓ 4.	Datastax Enterprise	Wide column, Multi-model	3.34	-0.07	-1.57
6.	6.	6.	Microsoft Azure Table Storage	Wide column	2.83	+0.03	-0.27
7.	7.	↑ 8.	Google Cloud Bigtable	Multi-model	2.65	0.00	-0.28
8.	8.	↓ 7.	Apache Accumulo	Wide column	2.35	+0.00	-0.73
9.	9.	9.	Amazon Keyspaces	Wide column	1.11	-0.04	+0.12
10.	10.	10.	HPE Ezmeral Data Fabric	Multi-model	0.87	+0.05	+0.04

Cassandra

open-source, distributed NoSQL database designed to handle massive amounts of data across many commodity servers while providing high availability and linear scalability with no single point of failure.

It is highly popular for applications that require constant uptime and must handle high-volume, continuous writes, such as IoT platforms, social media, and real-time analytics.

Most widely adopted wide column store.

Excellent detailed documentation.

Standard CQL (Cassandra Query Language) is SQL-like, making it accessible.

Cassandra

Design objectives:

- Full multi-primary database replication
- Global availability at low latency
- Scaling out on commodity hardware
- Linear throughput increase with each additional processor
- Online load balancing and cluster growth
- Partitioned key-oriented queries
- Flexible schema

organizes data within a cluster using:

Keyspace: Specifies the replication strategy for a dataset across different datacenters. Keyspaces serve as containers for tables.

Table: Tables are composed of rows and columns. Columns define the typed schema for a single datum in a table. Tables are partitioned based on the columns provided in the partition key. Can flexibly add new columns to tables with zero downtime.

Partition: the mandatory part of the primary key, used to identify the node in a cluster where the row is stored. All performant queries supply the partition key in the query.

Row: Contains a collection of columns identified by a unique primary key made up of the partition key and optionally additional clustering keys.

Column: A single datum with a type which belongs to a row.

Cassandra

CQL supports features over a partitioned dataset:

- Collection types including sets, maps, and lists
- User-defined types, tuples, functions and aggregates
- Storage-attached indexing (SAI) for secondary indexes
- Local secondary indexes (2i)
- User-defined types, functions and aggregates
- Single-partition lightweight transactions with atomic compare and set semantics
- (Experimental) materialized views

Cassandra explicitly chooses not to implement operations that require cross-partition coordination as they are typically slow and hard to provide highly available global semantics. Does not support:

- Cross-partition transactions
- Distributed joins
- Foreign keys or referential integrity

Dataset Partitioning

Consistent Hashing

Cassandra scales horizontally by hashing data into partitions and replicating them across nodes and failure domains. Since any replica can update any key, each key is versioned. Instead of Dynamo's vector clocks, Cassandra uses a simpler last-write-wins approach: every mutation, including deletes, is timestamped, and the newest value prevails. Effectively, each CQL row is an LWW-Element-Set CRDT for resolving conflicting replica updates.

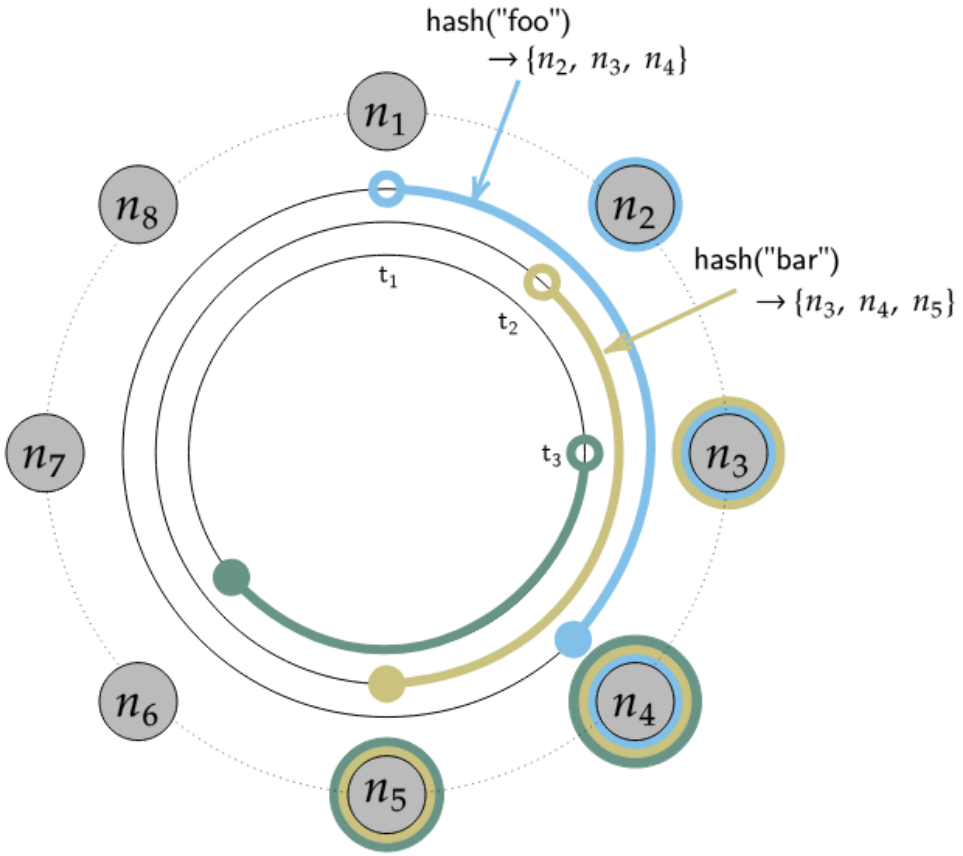
Token Ring

Cassandra distributes data using consistent hashing rather than simple modulo hashing. In naive hashing, adding a node forces most key-to-bucket mappings to change. Instead, Cassandra places nodes at token positions on a continuous hash ring and assigns each key to the next node on the ring, as in Chord. This approach ensures that when nodes are added or removed, only a small portion of keys must be moved.

Dataset Partitioning

$\text{token}(n_1) = t_1$
$\text{token}(n_2) = t_2$
$\text{token}(n_3) = t_3$
...

$\text{range}(t_1, t_2] \rightarrow \{n_2, n_3, n_4\}$
$\text{range}(t_2, t_3] \rightarrow \{n_3, n_4, n_5\}$
$\text{range}(t_3, t_4] \rightarrow \{n_4, n_5, n_6\}$
...



Tokens to Endpoints

Concepts

- *Token*: A single position on the Dynamo-style hash ring.
- *Endpoint*: A single physical IP and port on the network.
- *Host ID*: A unique identifier for a single "physical" node, usually present at one gEndpoint and containing one or more gTokens.
- *Virtual Node* (or vnode): A gToken on the hash ring owned by the same physical node, one with the same gHost ID.

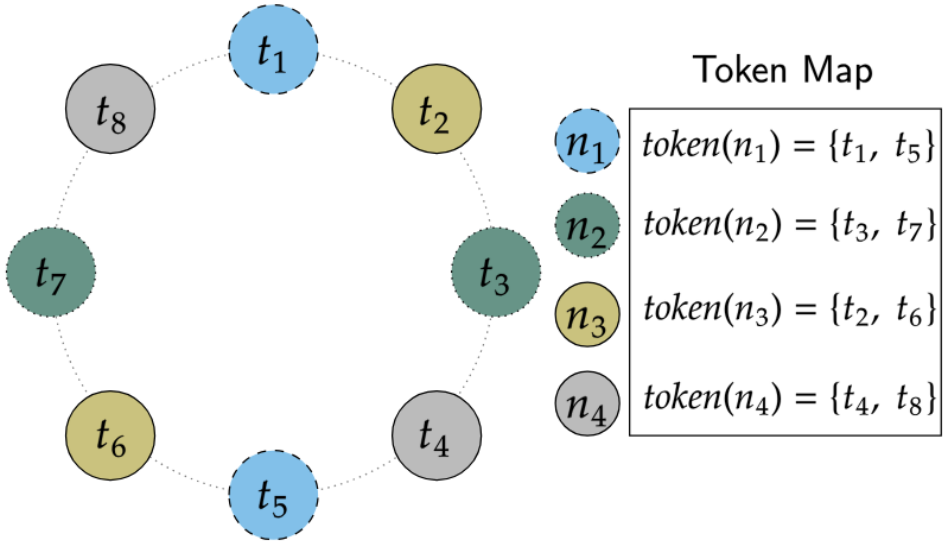
Cassandra's token map tracks which tokens on the hash ring belong to which physical nodes. By giving each node multiple tokens, a cluster can balance data more evenly - for example, four nodes can appear as eight logical positions.

Advantages:

- New nodes take on data evenly, improving cluster balance.
- Decommissioned nodes shed data evenly to others.
- If a node fails, token-aware queries spread load across many nodes.

Disadvantages:

- More tokens create more ring neighbors, increasing failure combination that can reduce availability.
- Cluster-wide tasks, like repairs, slow down as token counts grow.
- Operations spanning many token ranges may see degraded performance.



MultiMaster Replication

- **NetworkTopologyStrategy**:

set a separate replication factor for each datacenter and is preferred even for single-DC clusters, since it simplifies adding new datacenters later.

It also places replicas in different racks, based on the Snitch. If racks ≥ the DC's replication factor, all replicas come from distinct racks; otherwise, each rack gets at least one. This rack awareness can create uneven load—for example, a smaller rack may get more data, or a new single-node rack may receive replicas for the whole ring. To avoid this, many operators map all nodes in one availability zone to a single "rack."

- **SimpleStrategy**:

uses a single integer replication_factor to specify how many nodes store each row (e.g., 3 means three copies).

It ignores datacenters and racks, treating all nodes the same. To select replicas, Cassandra walks the ring from the target token forward, adding each unique node it encounters until it reaches the required number of replicas.

Data Versioning

- **Versioning**:

ensures eventual consistency using timestamp-based versioning. Each mutation gets a timestamp from the client or, if missing, the coordinator's clock, and conflicts resolve with last-write-wins—so accurate clock sync (e.g., NTP) is essential.

Every column in a row receives its own timestamp, and each resolves conflicts independently. Different primary keys in the same partition can therefore update without conflict. CQL collections like maps and sets use the same LWW approach, ensuring concurrent updates also reconcile cleanly.

- **Tunable Consistency**:

ONE: Only a single replica must respond.

TWO: Two replicas must respond.

THREE: Three replicas must respond.

QUORUM: A majority (n/2 + 1) of the replicas must respond.

ALL: All of the replicas must respond.

LOCAL_QUORUM: A majority of the replicas in the local datacenter (whichever datacenter the coordinator is in) must respond.

EACH_QUORUM: A majority of the replicas in each datacenter must respond.

LOCAL_ONE: Only a single replica must respond. In a multi-datacenter cluster, this also guarantees that read requests are not sent to replicas in a remote datacenter.

ANY: A single replica may respond, or the coordinator may store a hint. If a hint is stored, the coordinator will later attempt to replay the hint and deliver the mutation to the replicas. This consistency level is only accepted for write operations.

Gossip

Cassandra's gossip system spreads basic cluster state—membership, protocol versions, token info, and schema versions. Nodes exchange state about themselves and others, using (generation, version) clocks so outdated info can be ignored.

Each node gossips every second by:

- Updating its heartbeat (version) and local view of cluster state
- Contacting a random node
- Occasionally trying unreachable nodes
- Contacting a seed node if not already contacted

Seed nodes are special only during bootstrap: non-seed nodes must reach one to join the cluster, so clusters usually define several seeds per rack or datacenter. After bootstrap, seeds simply get more gossip traffic.

Gossip also drives control-plane actions: schema syncs when versions differ, and token metadata updates that teach nodes who owns which data.

Connecting

```
from cassandra.cluster import Cluster

# Connect to the cluster
cluster = Cluster(['192.168.0.1', '192.168.0.2'])
session = cluster.connect()

# Create Keyspace
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS test_db
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor':
3}
""")
session.set_keyspace('test_db')
```

Create Table

Scenario: Storing student grades. We want to query by Student ID, and order grades by Year.
Code Snippet:

```
CREATE TABLE student_grades (
    student_id UUID,
    year int,
    course_name text,
    grade float,
    PRIMARY KEY ((student_id), year, course_name)
) WITH CLUSTERING ORDER BY (year DESC);
```

Insert Data

Similar to SQL, but no referential integrity checks (Foreign Keys).
Code Snippet:

```
INSERT INTO student_grades (student_id, year, course_name, grade)
VALUES (123e4567-..., 2025, ‘CABD-CLOUDDB', 9.5);
```

Read Data

Queries must be restricted to partition key.
Code Snippet:

```
-- Efficient Query (Hits one node)
SELECT * FROM student_grades WHERE student_id = 123e4567-...;

-- Inefficient Query (Scans ALL nodes - Anti-pattern)
SELECT * FROM student_grades WHERE grade > 9; -- WILL FAIL or TIMEOUT
```

Update and Delete

Update: An update is just an Insert. It writes a new timestamped version.

Delete: Cassandra cannot immediately delete data from disk (SSTables are immutable).

Tombstone: A marker written to indicate data is deleted.

Issues: When deleting too much, reads slow down (skipping over tombstones).

Lightweight Transactions

Useful for "Check-and-Set" operations (e.g., creating a username that must be unique).
Uses Paxos consensus algorithm (Performance penalty: ~4x slower).

Code Snippet:

```
INSERT INTO users (username, password)
VALUES ('admin', '1234')
IF NOT EXISTS;
```

Collections

Cassandra supports Sets, Lists, and Maps within a single column.

Code Snippet:

```
CREATE TABLE users (
    id UUID PRIMARY KEY,
    emails set<text>,
    todo_list list<text>
);

UPDATE users SET emails = emails + {'work@test.com'} WHERE id = ...;
```

Tunable Consistency

Write Consistency:

- *ANY*: Written to hint handoff (fastest, risky).
- *QUORUM*: Written to 51% of replicas (balanced).

Read Consistency:

- *ONE*: Returns first response (fastest).
- *QUORUM*: Checks 51% of replicas and repairs data if mismatch (Read Repair).

Materialized Views

Automatic way to denormalize data for different query patterns.
Concept: Base table queries by ID. View queries by Email. Cassandra handles the sync.
Warning: Asynchronous. View update might lag behind Base table update.

Resources

What is Cassandra
Cassandra Architecture explained